MEMORY LAB: Making Linear Attention Visible

DataForge 2026 • Pathway Track — Explain the Frontier

Why this concept matters

Long-context AI systems face a practical tension: the model may need to use a growing history while hardware has finite memory and bandwidth. Standard self-attention makes token-to-token relationships explicit through an attention-score structure whose sequence dimension is quadratic. In autoregressive decoding, keys and values are also retained in a growing KV cache. The result is a system in which longer context is not free: more historical representations must be stored and moved.

The idea behind linear attention

Linear attention changes the organization of the computation. Rather than first constructing every query-key interaction, a feature-map formulation can regroup the operations so that key/value contributions are accumulated into a recurrent state. A compact educational form is $S_t = S_{(t-1)} + \phi(k_t)v_t$■. A query can then read the accumulated state. The important idea is not that every linear-attention method has exactly this equation; rather, the sequence history is represented through an evolving state instead of requiring the full pairwise score structure to be materialized. That reorganization can make sequence-length scaling more favorable, while changing retrieval behavior.

Why we built an interactive substrate

A formula alone is easy to misunderstand. Memory Lab therefore treats the concept as something the learner should manipulate. The 100-Token Race lets the learner change N and compare an N^2 pairwise-slot visualization with N incremental state updates. The core memory engine processes tokens one at a time and exposes the feature transform, rank-1 write, accumulated state and query readout. A prediction interaction asks the learner to estimate the scaling consequence before the computed result is shown. This follows a simple learning loop: observe → change a meaningful variable → predict → run → inspect → explain.

What is real and what is simplified

The browser code performs deterministic toy linear algebra rather than merely playing a pre-scripted animation. Its small feature dimension is deliberate: the state must remain visible enough for a learner to understand every update. The result should not be mistaken for a benchmark. It is not an official BDH implementation, not a production linear-attention kernel, and not evidence that one architecture is universally faster or more accurate. Animated presentation is used to make the computation legible; the underlying state changes are generated by the browser computation.

Where current research fits

Linear attention is an active family rather than a single finished architecture. CosFormer (ICLR 2022) explores cosine-based reweighting to improve linearized attention behavior. RetNet (2023) develops a retention mechanism that supports both parallel and recurrent computation and targets efficient long-sequence inference. Gated Linear Attention Transformers (2024) studies gating and hardware-efficient training, showing that practical systems concerns extend beyond asymptotic formulas. These works demonstrate a broader design space: researchers are changing feature maps, gates, state dynamics and hardware organization to balance memory efficiency, retrieval quality and speed.

Why BDH and BDH-CQ are relevant

The Pathway brief places BDH (Dragon Hatchling) in a post-Transformer family where memory and reasoning share a computational substrate. BDH-CQ connects contextual memory with attention, fast-weight memory and linear-attention views of contextual association. Memory Lab uses this connection to help learners see why an evolving state can be more than a computational trick: it can be interpreted as a form of short-term associative memory. The distinction is important. Our toy state is an explanatory lens, not an official reproduction of BDH or BDH-CQ. The artifact also avoids classifying BDH as a Mamba-style SSM, a distinction explicitly emphasized in the Pathway brief.

The trade-off we want the learner to remember

Compact memory is not perfect memory. A fixed-shaped state has finite information capacity. As more associations are written, different histories can interfere, retrieval can degrade, and older information can be overwritten or become harder to recover. In addition, asymptotic complexity does not automatically translate into lower wall-clock latency: kernel design, bandwidth, tensor dimensions, hardware tiling and implementation quality all matter. The educational goal is therefore not “linear attention is always better.” It is a more precise statement: reorganizing attention around incremental state updates can change the memory/computation trade-off, and the price can appear in retrieval fidelity, capacity and systems behavior.

What Memory Lab contributes

The contribution is a reusable teaching instrument rather than a new model architecture. It turns an abstract transition—pairwise comparison to incremental state—into a sequence of visible experiments. The learner can see the N^2 structure, change N, watch state updates accumulate, test a prediction, inspect the resulting matrix, connect the mechanism to BDH/BDH-CQ, and finish with limitations. This makes the central claim falsifiable at the level appropriate for a small educational substrate: if the state did not update incrementally or the scaling visualization did not follow the stated rules, the learner could see the discrepancy.

Closing perspective

The broader lesson is about how AI systems represent context. A model does not necessarily need to preserve every historical token in the same form in order to use context. It can instead maintain a learned or engineered state that summarizes what has happened so far. That shift opens a rich design space for recurrent memory, fast weights, gated state updates and post-Transformer architectures. It also creates a scientific question that remains important: how much information can a compact state preserve, how selectively can it retrieve that information, and how should fast contextual memory interact with slower durable parameters? Memory Lab is designed to make that question concrete enough to explore in under a minute.

Primary research and project sources

• Qin et al. (2022), *CosFormer: Rethinking Softmax in Attention*, ICLR 2022.

• Sun et al. (2023), *Retentive Network: A Successor to Transformer for Large Language Models*.

• Yang et al. (2024), *Gated Linear Attention Transformers with Hardware-Efficient Training*.

• Katharopoulos et al. (2020), *Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention*, foundational.

• Pathway Research (2026), Dragon Hatchling (BDH) and BDH-CQ technical materials supplied for the DataForge Pathway track.